No.	Python Basic Questions
	What is Python?
	Features of Python?
	Python Frameworks?
	Difference between Python 2 and Python 3
	What is an Interpreted language?
	What is a dynamically typed language?
	Is indentation required in python?
	File Extensions in Python?
	What is the difference between .py and .pyc files?
	What is PEP 8?
	Python Comments?
	What is Docstrings?
	how to get docstring in python
	Difference between Python comments and docstrings?

Ques. What is Python?

- Python is a high-level, interpreted, general-purpose programming language.
- Python is an **interpreter** language. It means it executes the code line by line, which helps in debugging and testing.
- It was created by **Guido van Rossum**, and released in **1991**
- It is used for:
    - web development (server-side)
    - software development
    - mathematics
    - system scripting

Ques. Features of Python?

1. **Easy:-** Python is very easy to learn and understand; using this Python tutorial, any beginner can understand the basics of Python.
2. **Interpreted:-** It is interpreted(executed) line by line. This makes it easy to test and debug.
3. **Object-Oriented:-** The Python programming language supports classes and objects. We discussed these above.
4. **Free and Open Source:-** The language and its source code are available to the public for free; there is no need to buy a costly license.
5. **Portable:-** Since it is open-source, you can run Python on Windows, Mac, Linux or any other platform. Your programs will work without needing to the changed for every machine.
6. **GUI Programming:-** You can use it to develop a GUI (Graphical User Interface). One way to do this is through Tkinter.
7. **Large Library:-** Python provides you with a large standard library. You can use it to implement a variety of functions without needing to reinvent the wheel every time. Just pick the code you need and continue. This lets you focus on other important tasks.

Ques. Python Frameworks?

- **Web Development:** Django, Pyramid, Bottle, Tornado, Flask, web2py
- **GUI Development:** tkInter, PyGObject, PyQt, PySide, Kivy, wxPython
- **Scientific and Numeric:** SciPy, Pandas, IPython
- **Software Development:** Buildbot, Trac, Roundup
- **System Administration:** Ansible, Salt, OpenStack, xonsh

Difference between Python 2 and Python 3?

Feature	Python 2	Python 3
Release Year	2000	2008
Current Status	❌ End of Life (Jan 2020)	☑ Actively maintained
Print Statement	print "Hello"	print("Hello")
Default String	Type ASCII	Unicode (UTF-8)
Integer Division	5 / 2 = 2	5 / 2 = 2.5
Input Function	raw_input()	input()
Range Function	range() returns list	range() returns iterator
Error Handling	Old syntax	Improved syntax
Libraries Support	Limited (deprecated)	Full & modern support
Performance	Slower	Faster & optimized
Security Updates	❌ No	☑ Yes

- Print Function

```
# Python 2
print "Hello World"

# Python 3
print("Hello World")
```

- Division Behavior

```
# Python 2
print(5 / 2)   # Output: 2

# Python 3
print(5 / 2)   # Output: 2.5
```

- Input Handling

```
# Python 2
name = raw_input("Enter name: ")

# Python 3
name = input("Enter name: ")
```

- Unicode vs String (Very Important)

```
# Python 2
s = "hello"     # ASCII
u = u"hello"    # Unicode


# Python 3
s = "hello"     # Unicode by default,  Remove u""
```

- range() Behavior Changed

```
# Python 2
range(5)    # returns list

# Python 3
range(5)    # returns iterator
```

- xrange() Removed

```
# Python 2
xrange(10)

# Python 3
range(10)
```

- Exception Syntax Changed

```
# Python 2
except Exception, e:
    print e

# Python 3
except Exception as e:
    print(e)
```

- long Type Removed

```
# Python 2
x = 123L

# Python 3
x = 123 # Python 3 handles large integers automatically.
```

Ques. What is an Interpreted language?

- An Interpreted language **executes** its statements **line by line**. Languages such as Python, Javascript, R, PHP, and Ruby are prime examples of Interpreted languages.

Ques. What is a dynamically typed language?

- Type-checking can be done at two stages:-

1. **Static-** Data Types are checked before execution.
2. **Dynamic-** Data Types are checked during execution. Python is an interpreted language, executes each statement line by line and thus type-checking is done on the fly, during execution. Hence, Python is a Dynamically Typed Language.

Ques. Is indentation required in python?

- Indentation in Python means the **spaces** (or **tabs**) you put at the beginning of a line of code.
- Indentation is necessary for Python. It specifies a block of code. All code within loops, classes, functions, etc is specified within an indented block.
- It is usually done using four space characters. If your code is not indented necessarily, it will not execute accurately and will throw errors as well.

Ques. File Extensions in Python?

- **.py–** The normal extension for a Python source file
- **.pyc-** The compiled bytecode
- **.pyd-** A Windows DLL file
- **.pyo-** A file created with optimizations
- **.pyw-** A Python script for Windows
- **.pyz-** A Python script archive

.py

- .py is the Python source-code file written by the developer, while .pyc is the cached bytecode generated by Python from the .py file. .pyc is executed by the Python Virtual Machine (PVM).`

.pyc

- It contains Python bytecode, which is an intermediate representation executed by the Python Virtual Machine.
- .pyc files are commonly stored inside the **pycache** directory:

```
project/
│
├── main.py
└── __pycache__/
    └── main.cpython-312.pyc
```

Why .pyc exists?

- Mainly to **avoid recompiling the source to bytecode every time**, when the cached bytecode is still valid. This can make module loading faster.

Ques. What is the difference between .py and .pyc files?

- We get bytecode after compilation of **.py** file (source code). **.pyc** files are not created for all the files that you run. It is only created for the files that you import.

`.py`	`.pyc`
.py files contain the source code of a program	**.pyc** file contains the bytecode of your program.
Human-readable	Mostly machine-readable/not intended for humans
We write our Python code here	Python generates it automatically
Example: `main.py`	Example: `main.cpython-312.pyc`
Executed after being compiled to bytecode	Bytecode is executed by Python Virtual Machine (PVM)

Ques. What is PEP 8?

- PEP stands for **Python Enhancement Proposal**.
- It is a set of rules that specify how to format Python code for maximum readability.
- PEP8 is a document that provides various guidelines to write the readable in python.
- PEP8 describe how the developers can write the beautiful code.

Ques. Python Comments?

- Comments are hints that we add to our code to make it easier to understand.
- Comments are short descriptions along with the code to increase its readability.
- **single Line Comments:-** Comments starts with a #, and Python will ignore them:

```
#This is a comment
print("Hello, World!")

print("Hello, World!") #This is a comment
```

Multi Line Comments(OR)Docstring

- To add a multiline comment you could insert a # for each line.

```
#This is a comment
#written in
#more than just one line
print("Hello, World!")
```

- you can add a multiline string (triple quotes) in your code, and place your comment inside it.

```
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

Ques. What is Docstrings?

- In Python, docstrings are a way of documenting modules, classes, functions, and methods. They are written within triple quotes (""" or ''') and can span multiple lines.
- Python docstrings are strings used right after the definition of a function, method, class, or module. They are used to document our code.
- There are two main types of docstrings:
    - **Single-line docstrings:** Used for simple explanations that fit on one line.
    - **Multi-line docstrings:** Used for more detailed explanations, including parameter descriptions, return values, and exceptions.

Ques. how to get docstring in python?

- **Example and Accessing Docstrings:-**

```
class Calculator:
    """
    A simple calculator class to perform basic arithmetic operations.
    """
    def add(self, a, b):
        """
        add(a, b): Return the sum of two numbers.
        """
        return a + b

    def multiply(self, a, b):
        """
        multiply(a, b): Return the product of two numbers.
        """
        return a * b

# Accessing the class docstring
print(Calculator.__doc__) # Output:- A simple calculator class to perform basic
arithmetic operations.

# Accessing the method docstring
print(Calculator.add.__doc__)     # Output:- add(a, b): Return the sum of two
numbers.
print(Calculator.multiply.__doc__)  # Output:-  multiply(a, b): Return the product
of two numbers.
```

```
def square(n):
    '''Take a number n and return the square of n.'''
    return n**2

print(square.__doc__)   # Output:- Take a number n and return the square of n.
```

- Using the **doc** attribute:

```
def my_function():
    """This is a docstring for my_function."""
    pass

print(my_function.__doc__)

Output:- This is a docstring for my_function.
```

- Using inspect.getdoc():

```
import inspect

def my_function():
    """
        This is a docstring for my_function
        with indentation.
    """
    pass

print(inspect.getdoc(my_function))

Output:-
This is a docstring for my_function
with indentation.
```

Ques. Difference between Python comments and docstrings?

- **Comments:** Comments in Python start with a hash mark (#) and are intended to explain the code to developers. They are **ignored** by the Python interpreter.
- **Docstrings:** Docstrings provide a description of the function, method, class, or module. Unlike comments, they are **not ignored** by the interpreter and can be accessed at runtime using the .**doc** attribute.